

Start coding or [generate](#) with AI.

```
import re # For regular expression operations, useful for text cleaning and pat
import string # Provides access to string constants like punctuation characters
from nltk.tokenize import word_tokenize, sent_tokenize # For splitting text int
from collections import Counter # For efficient frequency counting of items, us
import numpy as np # For numerical operations, essential for probability calcul

print("Libraries imported successfully.")
```

Libraries imported successfully.

```
sample_corpus = """Natural language processing (NLP) is a fascinating f

# Clean the text: remove extra whitespace, newlines, and ensure single
cleaned_corpus = re.sub(r'\s+', ' ', sample_corpus).strip()

print("Cleaned corpus sample (first 300 characters):")
print(cleaned_corpus[:300])
```

ned corpus sample (first 300 characters):
ral language processing (NLP) is a fascinating field at the intersection of compu

```
lowercase_corpus = cleaned_corpus.lower()
print("Corpus converted to lowercase (first 300 characters):")
print(lowercase_corpus[:300])
```

Corpus converted to lowercase (first 300 characters):
natural language processing (nlp) is a fascinating field at the intersection of

```
def remove_punctuation_and_numbers(text):
    # Remove any character that is not a letter or a space
    return re.sub(r'^a-zA-Z\s', '', text)

text_without_punc_nums = remove_punctuation_and_numbers(lowercase_corpus)
print("Corpus after removing punctuation and numbers (first 300 characters):")
print(text_without_punc_nums[:300])
```

Corpus after removing punctuation and numbers (first 300 characters):
natural language processing nlp is a fascinating field at the intersection of co

```
import nltk
nltk.download('punkt')
print("NLTK 'punkt' resource downloaded successfully.")
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
NLTK 'punkt' resource downloaded successfully.
```

```
import nltk
nltk.download('punkt_tab')

def tokenize_and_add_sentence_markers(text):
    # Split text into sentences
    sentences = sent_tokenize(text)
    processed_sentences = []
    for sentence in sentences:
        # Tokenize each sentence into words
        words = word_tokenize(sentence)
        # Add start and end tokens
        processed_sentences.append(['<s>'] + words + ['</s>'])
    return processed_sentences

processed_sentences = tokenize_and_add_sentence_markers(text_without_punc_nums)
print("First two processed sentences:")
for i, sentence in enumerate(processed_sentences[:2]):
    print(f"Sentence {i+1}: {sentence}")
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
First two processed sentences:
Sentence 1: ['<s>', 'natural', 'language', 'processing', 'nlp', 'is', 'a', 'fasc
```

```
from itertools import tee

def generate_ngrams(sentences, n):
    """
    Generates n-grams from a list of processed sentences.
    Each sentence is expected to be a list of words including start/end tokens.
    """
    all_ngrams = []
    for sentence in sentences:
        if len(sentence) < n: # Skip sentences too short to form n-grams of spe
            continue
        # Generate n-grams using a sliding window approach
        for i in range(len(sentence) - n + 1):
            all_ngrams.append(tuple(sentence[i:i+n]))
    return all_ngrams

# Generate unigrams (n=1)
unigrams = generate_ngrams(processed_sentences, 1)
unigram_counts = Counter(unigrams)

# Generate bigrams (n=2)
bigrams = generate_ngrams(processed_sentences, 2)
```

```

bigram_counts = Counter(bigrams)

# Generate trigrams (n=3)
trigrams = generate_ngrams(processed_sentences, 3)
trigram_counts = Counter(trigrams)

print("Top 5 Unigrams:")
for unigram, count in unigram_counts.most_common(5):
    print(f"{unigram}: {count}")

print("\nTop 5 Bigrams:")
for bigram, count in bigram_counts.most_common(5):
    print(f"{bigram}: {count}")

print("\nTop 5 Trigrams:")
for trigram, count in trigram_counts.most_common(5):
    print(f"{trigram}: {count}")

```

Top 5 Unigrams:

```

('and',): 7
('the',): 5
('nlp',): 4
('a',): 4
('text',): 4

```

Top 5 Bigrams:

```

('<s>', 'natural'): 1
('natural', 'language'): 1
('language', 'processing'): 1
('processing', 'nlp'): 1
('nlp', 'is'): 1

```

Top 5 Trigrams:

```

('<s>', 'natural', 'language'): 1
('natural', 'language', 'processing'): 1
('language', 'processing', 'nlp'): 1
('processing', 'nlp', 'is'): 1
('nlp', 'is', 'a'): 1

```

```

total_words = sum(unigram_counts.values())
unigram_probabilities = {unigram: count / total_words for unigram, count in uni

print("Top 5 Unigram Probabilities:")
for unigram, prob in sorted(unigram_probabilities.items(), key=lambda item: ite
    print(f"{unigram}: {prob:.6f}")

```

Top 5 Unigram Probabilities:

```

('and',): 0.053846
('the',): 0.038462
('nlp',): 0.030769
('a',): 0.030769
('text',): 0.030769

```

```

from collections import defaultdict

vocabulary = set([word[0] for word in unigram_counts.keys()]) # Get all unique
vocabulary_size = len(vocabulary)

def calculate_bigram_probabilities(bigram_counts, unigram_counts, vocabulary_size):
    bigram_probabilities = defaultdict(float)

    # Iterate through all possible context words (first word of a bigram)
    # unigram_counts stores words as tuples, so we iterate its keys.
    for word1_tuple in unigram_counts.keys(): # word1_tuple is like ('word',)
        word1_str = word1_tuple[0] # Extract the string form of the word, e.g.,

        # Get count of the context word (w_i-1) using the tuple key
        context_count = unigram_counts[word1_tuple]

        # Iterate through all possible second words (word2) in the vocabulary
        for word2_str in vocabulary: # word2_str is like 'word'
            bigram = (word1_str, word2_str) # Form the bigram tuple

            # Get the count of the bigram. If not present, Counter returns 0.
            bigram_occurrence_count = bigram_counts[bigram]

            # Apply Add-one smoothing formula:  $P(\text{word2} \mid \text{word1}) = \frac{\text{Count}(\text{word1}, \text{word2}) + 1}{\text{Count}(\text{word1}) + \text{vocabulary\_size}}$ 
            probability = (bigram_occurrence_count + 1) / (context_count + vocabulary_size)
            bigram_probabilities[bigram] = probability

    return bigram_probabilities

bigram_probabilities = calculate_bigram_probabilities(bigram_counts, unigram_counts, vocabulary_size)

print("Top 5 Bigram Probabilities with Add-one Smoothing:")
# Sort by probability in descending order and print top 5
for bigram, prob in sorted(bigram_probabilities.items(), key=lambda item: item[1]):
    print(f"{bigram}: {prob:.6f}")

```

```

Top 5 Bigram Probabilities with Add-one Smoothing:
('<s>', 'natural'): 0.019802
('natural', 'language'): 0.019802
('processing', 'nlp'): 0.019802
('is', 'a'): 0.019802
('fascinating', 'field'): 0.019802

```

```

def calculate_trigram_probabilities(trigram_counts, bigram_counts, vocabulary_size):
    trigram_probabilities = defaultdict(float)

    # Iterate through all possible context bigrams (w_i-2, w_i-1)
    for bigram_tuple in bigram_counts.keys(): # bigram_tuple is like ('word1', 'word2')
        word1_str, word2_str = bigram_tuple[0], bigram_tuple[1]

        # Get count of the context bigram (w_i-2, w_i-1) using the tuple key

```

```

context_bigram_count = bigram_counts[bigram_tuple]

# Only proceed if the context bigram was observed at least once (before
# If context_bigram_count is 0, then the denominator (context_bigram_cc
# but for an actual occurrence, we need bigram_counts to be non-zero.
# However, due to bigram smoothing, all bigrams now have a non-zero 'pr
# We need the *raw count* for the denominator, so we use bigram_counts,

# If the context bigram itself was not observed (count is 0), its smoot
# would still be used in a real model for  $P(w_2|w_1)$ , but here for trigram
# If context_bigram_count is 0, this denominator term for raw count is

# Ensure bigram_counts is used correctly for the raw count of the conte
# The formula is  $(\text{Count}(w_1, w_2, w_3) + 1) / (\text{Count}(w_1, w_2) + V)$ 

# Iterate through all possible third words (word3) in the vocabulary
for word3_str in vocabulary:
    trigram = (word1_str, word2_str, word3_str) # Form the trigram tuple

    # Get the count of the trigram. If not present, Counter returns 0.
    trigram_occurrence_count = trigram_counts[trigram]

    # Apply Add-one smoothing formula:  $P(\text{word3} | \text{word1}, \text{word2}) = (\text{Count}$ 
    # The denominator should be the count of the prefix bigram (word1,

    # The `bigram_counts` variable already holds the raw counts for the
    # We must use the raw count here, not the smoothed probability.
    denominator = context_bigram_count + vocabulary_size

    # Avoid division by zero if context bigram was never observed (and
    # In add-one smoothing, if context_bigram_count is 0, denominator b
    if denominator == 0:
        # This case should ideally not happen if vocabulary_size > 0
        probability = 0.0 # Or some minimal value, but  $V > 0$  usually
    else:
        probability = (trigram_occurrence_count + 1) / denominator

    trigram_probabilities[trigram] = probability

return trigram_probabilities

trigram_probabilities = calculate_trigram_probabilities(trigram_counts, bigram_

print("Top 5 Trigram Probabilities with Add-one Smoothing:")
# Sort by probability in descending order and print top 5
for trigram, prob in sorted(trigram_probabilities.items(), key=lambda item: ite
    print(f"{trigram}: {prob:.6f}")

```

```

Top 5 Trigram Probabilities with Add-one Smoothing:
('<s>', 'natural', 'language'): 0.019802
('natural', 'language', 'processing'): 0.019802
('language', 'processing', 'nlp'): 0.019802

```

```
(('processing', 'nlp', 'is')): 0.019802
(('nlp', 'is', 'a')): 0.019802
```

```
test_sentences_raw = [
    "Natural language processing is an interesting field.",
    "NLP has seen many advancements.",
    "This is a test sentence for the language model.",
    "Human language generation is a complex task.",
    "Language models predict the next word."
]

def preprocess_sentence(sentence_text):
    lowercase_sentence = sentence_text.lower()
    without_punc_nums = remove_punctuation_and_numbers(lowercase_sentence)
    # The tokenize_and_add_sentence_markers function expects a single string
    # and returns a list of sentences (even if there's only one).
    # We want to get the first processed sentence from the list.
    tokenized_sentence = tokenize_and_add_sentence_markers(without_punc_nums)[0]
    return tokenized_sentence

processed_test_sentences = [preprocess_sentence(s) for s in test_sentences_raw]

print("First two processed test sentences:")
for i, sentence in enumerate(processed_test_sentences[:2]):
    print(f"Sentence {i+1}: {sentence}")
```

```
First two processed test sentences:
Sentence 1: ['<s>', 'natural', 'language', 'processing', 'is', 'an', 'interesting', 'field', '</s>']
Sentence 2: ['<s>', 'nlp', 'has', 'seen', 'many', 'advancements', '</s>']
```

```
def get_unigram_sentence_prob(sentence, unigram_probabilities, total_words, vocab_size):
    # For unigrams, we apply smoothing similarly.  $P(\text{word}) = (\text{Count}(\text{word}) + 1) / (\text{total\_words} + \text{vocab\_size})$ 
    # However, the unigram_probabilities dictionary already stores the unsmoothed probabilities
    # If we need smoothing for unigrams (not explicitly asked but good for robustness)
    # we would apply a similar formula. For now, we'll use the pre-calculated unigram probabilities
    # For unseen words in the unigram model, we will use a small constant for the probability
    # to avoid zero probability for the entire sentence.

    sentence_prob = 1.0
    # Use a very small probability for unseen unigrams if not using explicit unigram probabilities
    #  $(1 / (\text{total\_words} + \text{vocab\_size}))$  is a good estimate for smoothed unigram probabilities
    # Let's consider  $1e-10$  as a fallback for words not in our unigram_probabilities dictionary
    min_unigram_prob = 1 / (total_words + vocab_size) # Smoothed probability for unseen words

    for word in sentence:
        word_tuple = (word,)
        if word_tuple in unigram_probabilities:
            sentence_prob *= unigram_probabilities[word_tuple]
        else:
            # If a word is completely unseen even after building unigram counts
```

```

        # This case means the word is not even in the vocabulary used for `
        # We use a smoothed probability for such entirely new words.
        sentence_prob *= min_unigram_prob

    return sentence_prob

def get_bigram_sentence_prob(sentence, bigram_probabilities, unigram_probabilit
    sentence_prob = 1.0
    #  $P(w_1, w_2, \dots, w_n) = P(w_1) * P(w_2|w_1) * P(w_3|w_2) * \dots * P(w_n|w_{n-1})$ 
    # For  $P(w_1)$ , we use  $P(w_1|<s>)$ .
    # For other terms,  $P(w_i|w_{i-1})$ .

    # Handle the probability of the first word  $P(w_1|<s>)$ 
    # The first word in our processed sentence is always '<s>'. The second word
    # So, for the bigram model, the sequence starts effectively from '<s> w1'.
    # The first term is  $P(\text{word}[1]|\text{word}[0])$  where  $\text{word}[0]$  is '<s>'.
    # This means the sentence starts with '<s>' followed by the actual first wo
    # We calculate  $P(w_i | w_{i-1})$ 

    # We need to account for the initial <s> token and the actual words
    # The sentence here includes <s> and </s>, so it is ['<s>', 'word1', 'word2

    #  $P(w_1, w_2, \dots, w_n) \text{ approx} = P(w_1|<s>) * P(w_2|w_1) * \dots * P(</s>|w_n)$ 

    # The bigram_probabilities dict already contains  $P(\text{word2} | \text{word1})$  with add-

    for i in range(1, len(sentence)): # Start from the actual first word after
        prev_word = sentence[i-1]
        current_word = sentence[i]
        bigram = (prev_word, current_word)

        if bigram in bigram_probabilities:
            prob = bigram_probabilities[bigram]
        else:
            # This case should ideally not happen if calculate_bigram_probabili
            # However, if 'prev_word' was not in our original unigram vocabular
            # which could lead to issues. Assuming all words in `processed_test
            # Fallback to smoothed probability for unseen bigram ( $w_{i-1}, w_i$ ) i
            # This happens if 'prev_word' is not in unigram_counts.keys() when

            # We need the count of the previous word to calculate the smoothed
            # If prev_word is not in unigram_counts, then context_count would b
            # This implies the current approach where `calculate_bigram_probabi
            # So, theoretically, we should not hit this 'else' if our `calculat

            # If we were to calculate on demand for an unseen bigram:
            prev_word_tuple = (prev_word,)
            context_count = unigram_counts.get(prev_word_tuple, 0) # Get count,

            if context_count == 0: # If prev_word was not seen in training
                prob = 1 / (vocabulary_size) # Default minimal probability, ess

```

```

        else:
            prob = 1 / (context_count + vocabulary_size) # Smoothed prob fc

        sentence_prob *= prob

    return sentence_prob

def get_trigram_sentence_prob(sentence, trigram_probabilities, bigram_counts, v
    sentence_prob = 1.0
    #  $P(w_1, w_2, \dots, w_n) = P(w_1 | \langle s \rangle) * P(w_2 | \langle s \rangle, w_1) * P(w_3 | w_1, w_2) * \dots * P(w_n |$ 

    # The sentence includes  $\langle s \rangle$  and  $\langle /s \rangle$ , so it is ['<s>', 'word1', 'word2', ..
    # We need to consider three cases for the initial probabilities and then th
    # 1.  $P(\text{word1} | \langle s \rangle)$ 
    # 2.  $P(\text{word2} | \langle s \rangle, \text{word1})$ 
    # 3.  $P(\text{word}_i | \text{word}_{\{i-2\}}, \text{word}_{\{i-1\}})$  for  $i \geq 2$  (using 0-indexed sentenc

    # Note: Our `processed_sentences` start with '<s>'. So, sentence[0] is '<s>
    # The first word is sentence[1], the second is sentence[2], etc.

    # Handle  $P(\text{word1} | \langle s \rangle)$  using the bigram model (smoothed bigram_probabiliti
    # This logic is simpler: treat the sentence from '<s>' onwards as a sequenc
    # The trigram model starts effectively with '<s> <s> w1' or '<s> w1 w2'.
    # For our purpose, the first word probability  $P(w_1)$  and  $P(w_2 | w_1)$  are often
    # or special handling. Let's follow the instruction's suggestion:
    # "For  $P(w_1)$  and  $P(w_2 | w_1)$  in trigram model, assume  $P(w_1 | \langle s \rangle)$  and  $P(w_2 | \langle s \rangle,$ 

    # Let's consider `sentence` as ['<s>', 'w1', 'w2', ..., 'wn', '</s>']
    # First term:  $P(w_1 | \langle s \rangle)$ . This is a bigram prob.
    # We need bigram_probabilities. The current bigram_probabilities contains P
    # The bigram ( $\langle s \rangle$ , sentence[1]) would give  $P(\text{sentence}[1] | \langle s \rangle)$ .

    # Second term:  $P(w_2 | \langle s \rangle, w_1)$ . This is a trigram prob (trigram: ( $\langle s \rangle$ , w1, w2
    # If the sentence has only one actual word (e.g., ['<s>', 'w1', '</s>'])
    # Then only the first term applies ( $P(w_1 | \langle s \rangle) * P(\langle /s \rangle | w_1)$ ).
    # If the sentence has zero actual words, this function wouldn't be called.

    # We iterate from the third word (index 2) onwards, forming trigrams.
    # The terms are:  $P(\text{sentence}[1] | \text{sentence}[0]) * P(\text{sentence}[2] | \text{sentence}[0], \text{se}$ 

    #  $P(\text{word1} | \langle s \rangle)$ 
    if len(sentence) >= 2: # At least ['<s>', 'word1']
        bigram_for_initial_prob = ('<s>', sentence[1])
        # Use the smoothed bigram probability from bigram_probabilities
        sentence_prob *= bigram_probabilities.get(bigram_for_initial_prob, 1 /
    else:
        # Sentence is just ['<s>']. Should not happen with valid input, but for
        return 0.0

    #  $P(\text{word2} | \langle s \rangle, \text{word1})$ 
    if len(sentence) >= 3: # At least ['<s>', 'word1', 'word2']

```



```

    trigram_for_second_prob = ('<s>', sentence[1], sentence[2])
    # Use the smoothed trigram probability from trigram_probabilities
    sentence_prob *= trigram_probabilities.get(trigram_for_second_prob, 1 /

# P(word_i | word_{i-2}, word_{i-1}) for i >= 3
for i in range(3, len(sentence)): # Start from the actual third word (index
    prev_prev_word = sentence[i-2]
    prev_word = sentence[i-1]
    current_word = sentence[i]
    trigram = (prev_prev_word, prev_word, current_word)

    if trigram in trigram_probabilities:
        prob = trigram_probabilities[trigram]
    else:
        # Fallback for an unseen trigram. Calculate smoothed prob on demand
        context_bigram = (prev_prev_word, prev_word)
        context_count = bigram_counts.get(context_bigram, 0)
        if context_count == 0:
            prob = 1 / (vocabulary_size) # Default minimal probability, essentially
        else:
            prob = 1 / (context_count + vocabulary_size) # Smoothed probability for

    sentence_prob *= prob

return sentence_prob

print("Probability calculation functions defined.")

```

Probability calculation functions defined.

```

print("\n--- Sentence Probabilities ---")
for i, sentence in enumerate(processed_test_sentences):
    original_sentence = test_sentences_raw[i]
    print(f"\nOriginal Sentence {i+1}: {original_sentence}")
    print(f"Processed Sentence: {sentence}")

    # Calculate Unigram Probability
    unigram_prob = get_unigram_sentence_prob(sentence, unigram_probabilities, test_sentences_raw)
    print(f" Unigram Model Probability: {unigram_prob:.15f}")

    # Calculate Bigram Probability
    bigram_prob = get_bigram_sentence_prob(sentence, bigram_probabilities, test_sentences_raw)
    print(f" Bigram Model Probability: {bigram_prob:.15f}")

    # Calculate Trigram Probability
    trigram_prob = get_trigram_sentence_prob(sentence, trigram_probabilities, test_sentences_raw)
    print(f" Trigram Model Probability: {trigram_prob:.15f}")

print("\nInterpretation: A lower probability means the sentence is less likely

```

--- Sentence Probabilities ---

Original Sentence 1: Natural language processing is an interesting field.

Processed Sentence: ['<s>', 'natural', 'language', 'processing', 'is', 'an', 'in

Unigram Model Probability: 0.0000000000000000

Bigram Model Probability: 0.0000000000000001

Trigram Model Probability: 0.0000000000000001

Original Sentence 2: NLP has seen many advancements.

Processed Sentence: ['<s>', 'nlp', 'has', 'seen', 'many', 'advancements', '</s>']

Unigram Model Probability: 0.0000000000000004

Bigram Model Probability: 0.0000000000003696

Trigram Model Probability: 0.0000000000001941

Original Sentence 3: This is a test sentence for the language model.

Processed Sentence: ['<s>', 'this', 'is', 'a', 'test', 'sentence', 'for', 'the',

Unigram Model Probability: 0.0000000000000000

Bigram Model Probability: 0.0000000000000000

Trigram Model Probability: 0.0000000000000000

Original Sentence 4: Human language generation is a complex task.

Processed Sentence: ['<s>', 'human', 'language', 'generation', 'is', 'a', 'compl

Unigram Model Probability: 0.0000000000000000

Bigram Model Probability: 0.0000000000000000

Trigram Model Probability: 0.0000000000000000

Original Sentence 5: Language models predict the next word.

Processed Sentence: ['<s>', 'language', 'models', 'predict', 'the', 'next', 'wor

Unigram Model Probability: 0.0000000000000000

Bigram Model Probability: 0.0000000000000070

Trigram Model Probability: 0.0000000000000038

Interpretation: A lower probability means the sentence is less likely according

```
def calculate_perplexity(sentence_prob, num_words):
    """
    Calculates the perplexity of a sentence given its probability and number of
    Perplexity = (1 / Sentence_Probability)^(1/N)
    """
    if sentence_prob == 0:
        return float('inf') # Infinite perplexity for zero probability

    # To avoid overflow with very small probabilities, it's often better to wor
    # log(P) * (-1/N) = log(Perplexity)
    # Perplexity = exp(log(P) * (-1/N))
    try:
        # Ensure the probability is not negative or zero if it somehow became s
        if sentence_prob <= 0:
            return float('inf')
        # Use numpy.power for robustness with small numbers
        perplexity = np.power(1 / sentence_prob, 1 / num_words)
```

```

except OverflowError:
    # Handle cases where 1/sentence_prob is too large for float
    # This might happen if sentence_prob is extremely close to zero.
    # In such cases, perplexity is effectively infinite.
    perplexity = float('inf')
return perplexity

print("Perplexity calculation function defined.")

```

Perplexity calculation function defined.

```

print("\n--- Sentence Perplexities ---")
perplexities_data = []

for i, sentence in enumerate(processed_test_sentences):
    original_sentence = test_sentences_raw[i]
    # N = number of actual words, excluding '<s>' and '</s>'
    num_words = len(sentence) - 2

    # Calculate Unigram Probability and Perplexity
    unigram_prob = get_unigram_sentence_prob(sentence, unigram_probabilities, t
    unigram_perplexity = calculate_perplexity(unigram_prob, num_words)

    # Calculate Bigram Probability and Perplexity
    bigram_prob = get_bigram_sentence_prob(sentence, bigram_probabilities, unig
    bigram_perplexity = calculate_perplexity(bigram_prob, num_words)

    # Calculate Trigram Probability and Perplexity
    trigram_prob = get_trigram_sentence_prob(sentence, trigram_probabilities, t
    trigram_perplexity = calculate_perplexity(trigram_prob, num_words)

    print(f"\nOriginal Sentence {i+1}: {original_sentence}")
    print(f"  Number of words (N): {num_words}")
    print(f"  Unigram Perplexity: {unigram_perplexity:.2f}")
    print(f"  Bigram Perplexity: {bigram_perplexity:.2f}")
    print(f"  Trigram Perplexity: {trigram_perplexity:.2f}")

    perplexities_data.append({
        "sentence": original_sentence,
        "unigram_perplexity": unigram_perplexity,
        "bigram_perplexity": bigram_perplexity,
        "trigram_perplexity": trigram_perplexity
    })

print("\nPerplexity calculations complete.")

```

--- Sentence Perplexities ---

Original Sentence 1: Natural language processing is an interesting field.
 Number of words (N): 7

```
Unigram Perplexity: 525.49  
Bigram Perplexity: 145.08  
Trigram Perplexity: 144.27
```

Original Sentence 2: NLP has seen many advancements.

```
Number of words (N): 5  
Unigram Perplexity: 773.87  
Bigram Perplexity: 193.40  
Trigram Perplexity: 219.98
```

Original Sentence 3: This is a test sentence for the language model.

```
Number of words (N): 9  
Unigram Perplexity: 222.21  
Bigram Perplexity: 146.16  
Trigram Perplexity: 167.36
```

Original Sentence 4: Human language generation is a complex task.

```
Number of words (N): 7  
Unigram Perplexity: 467.68  
Bigram Perplexity: 160.63  
Trigram Perplexity: 193.89
```

Original Sentence 5: Language models predict the next word.

```
Number of words (N): 6  
Unigram Perplexity: 373.60  
Bigram Perplexity: 155.88  
Trigram Perplexity: 172.14
```

Perplexity calculations complete.